



Symphony6 Security Drivers User Manual for STBMs

Document Number:

Revision Number: 1.0

Revision Date: May 09, 2025

Montage LZ Technology Proprietary and Confidential

Revision History

Revision	Revision Date	Changes	
		Page Number	Description
1.0	May 09, 2025	-	Initial Release

1 Contents

1	Overview	4
2	Architecture of Security Drivers	4
3	Thread/Process Safety Mechanisms Used.....	4
4	API Calling Conventions and Notifications	5
5	Notifications for OTP Operations	6
6	Hardware Resources Usages	7
7	Explanations of Internal Structures	7
7.1	SCertResourceHandle	7
7.2	SSecCipherSession	8
7.3	SSecStreamSession	9
7.4	SSecHashContext	10
7.5	SCsdInitParameters	11
7.6	SCsdTerminateParameters	11
7.7	ECsdBootMode	11
7.8	ECsdScsSize	12
7.9	SCsdScsPvPathHandle.....	12
7.10	SCsdScrKeyPathHandle	12
7.11	SCsdDscKeyPathHandle	12
7.12	SCsdR2RKeyPathHandle	13
7.13	External CSD Montage-LZ API.....	13
7.14	. External SEC Montage-LZ API.....	13
8	Examples	14
8.1	RAM2RAM Encryption/Decryption Example	14
8.2	Transport Stream Descrambling Example	15
9	Acronyms	17

1 Overview

This is the user manual of security drivers for Symphony6 which is implemented to suffice NOCS3.2 requirements. This manual is for STBMs who are going to integrate Symphony6 into their NASC compliant devices.

Security drivers described in this document are used to implement NOCS3.2 APIs, including:

- Overall architecture of the drivers
- Mechanisms used to ensure thread/process safety
- API calling conventions, and notifications
- Hardware resources usages
- Internal structures used by security drivers

For usage of STR related drivers, please refer Montage other documents released along with SDK.

2 Architecture of Security Drivers

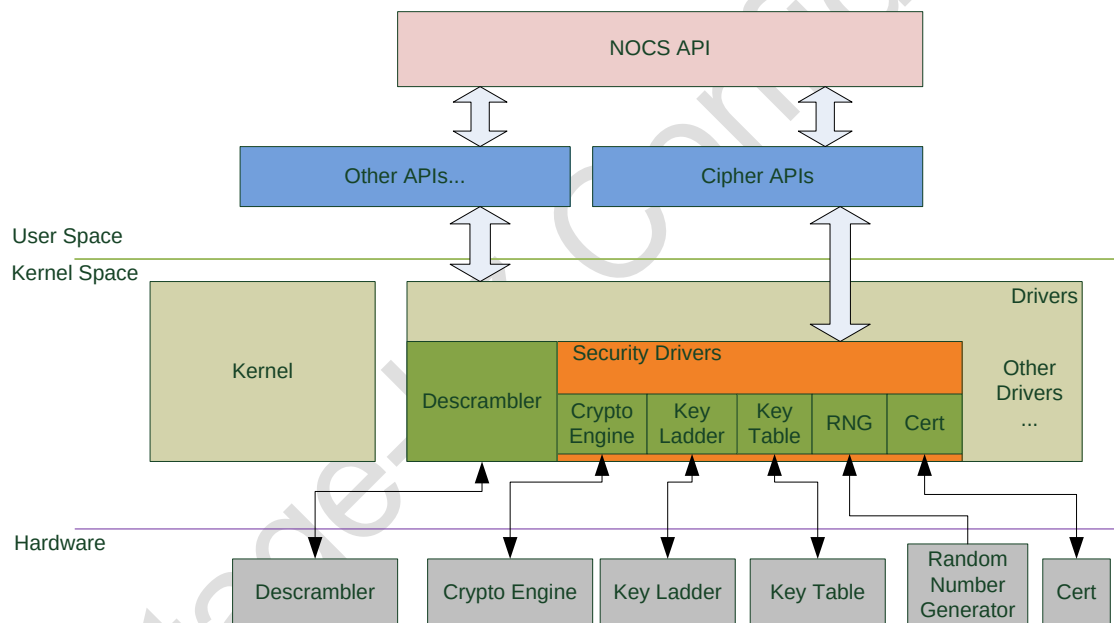


Figure 1 Overall Architecture of Linux based Security Drivers

In Linux platform based SDK released to STBMs, security drivers are separated into 2 parts:

- Kernel space drivers, implemented as Linux Device Drivers, communicating with hardware.
- User space APIs, providing user friendly APIs. Any call to security features should call through these cipher APIs.

3 Thread/Process Safety Mechanisms Used

All calling to security drivers shall call through cipher APIs.

Cipher APIs are locked by pthreads mutex, so they are thread safety. Multi-process call is not supported.

4 API Calling Conventions and Notifications

STBMs shall not call security drivers directly resided in kernel space, but call cipher APIs provided by SDK.

Security drivers are composed of five components:

- **Key Table**
Key Table driver is abstraction of hardware mechanism to store keys and its related properties. It has 128 key slots. Slots 4~124 are for common purpose, slot 1 is for flash protection key only, slot 0, 1~3 are used for other specific purpose what STBMs don't need to care. In NOCS3.2 chipset, keys in key slot cannot be read by software.
- **Key Ladder**
Key Ladder driver is abstraction of key ladder hardware. Its result is exported to a software specified key slot, and the data also cannot be read out.
- **Crypto Engine**
Crypto Engine driver is abstraction of memory-to-memory hardware crypto engine.
- **RNG**
RNG stands for random number generator. It is a hardware mechanism, produces true random numbers.
- **CERT**
CERT is an IP owned by NAGRA. Please refer relevant information released by NAGRA.

All five components are encapsulated by cipher APIs. When starting to use these APIs, please refer to [Example section](#) below as reference. For more details, please refer to the cipher API document released with SDK.

In NOCS3.2, a set of EMIs are defined. For Symphony6, supported EMIs are listed below.

#define NOCS_EMI_MPEG_TS_DVB_CSA2	0x0000
#define NOCS_EMI_MPEG_TS_DVB_CSA3	0x0001
#define NOCS_EMI_MPEG_TS_DVB_ASA_64	0x0010
#define NOCS_EMI_MPEG_TS_DVB_ASA_128	0x0011
#define NOCS_EMI_MPEG_TS_DVB_ASA_LIGHT	0x0012
#define NOCS_EMI_MPEG_TS_DVB_AES128_IDSA	0x0020
#define NOCS_EMI_MPEG_TS_DVB_AES128_ECB_TAIL_CLEAR	0x0021
#define NOCS_EMI_MPEG_TS_DVB_AES128_CBC_TAIL_CLEAR	0x0022
#define NOCS_EMI_MPEG_TS_DVB_AES128_CISSA_V1	0x0023
#define NOCS_EMI_MPEG_TS_DVB_DES_ECB_CTS	0x0031
#define NOCS_EMI_MPEG_TS_DVB_TDES_CBC_ZEROIV_DVS042	0x0040
#define NOCS_EMI_MPEG_TS_DVB_TDES_ECB_TAIL_CLEAR	0x0041
#define NOCS_EMI_MPEG_TS_DVB_MULT12	0x0050
#define NOCS_EMI_AES128_CBC_ZEROIV_TAIL_CLEAR	0x4020
#define NOCS_EMI_AES128_ECB_TAIL_CLEAR	0x4021
#define NOCS_EMI_AES128_MPEG_DASH_CBC_CLEAR_PADDING	0x4022
#define NOCS_EMI_AES128_CBC_PKCS7_PADDING	0x4023
#define NOCS_EMI_AES128_MPEG_DASH_CTR	0x4024
#define NOCS_EMI_AES128_CBC_TAIL_CLEAR	0x4026
#define NOCS_EMI_AES128_CTR	0x4027
#define NOCS_EMI_AES128_GCM	0x4028
#define NOCS_EMI_AES128_HLS	0x4029
#define NOCS_EMI_AES128_MPEG_DASH_CBCS	0x402A
#define NOCS_EMI_AES128_MPEG_DASH_CTR_CENS	0x402B
#define NOCS_EMI_AES256_ECB_TAIL_CLEAR	0x4030
#define NOCS_EMI_TDES_CBC_ZEROIV_TAIL_CLEAR	0x4040
#define NOCS_EMI_TDES_ECB_TAIL_CLEAR	0x4041
#define NOCS_EMI_TDES_CBC_TAIL_CLEAR	0x4043
#define NOCS_EMI_HMAC_SHA256	0x4060

For descriptions of EMI, please refer to related documents released by NAGRA.
 Cipher APIs are designed to handle data/keys based on the capabilities of Symphony6 chipset.
 So here are some tips have to be notified:

- Input/Output Buffer

While handling big data buffer, a continuity physical input/output buffer is required. It is recommended to use MMZ API to allocate such kind of buffer. For example, code fragment below is to show how to use MMZ API to allocate 64K bytes continuity data buffer.

For more details about MMZ, please refer related SDK documents.
 The same input and output buffer is allowed. If so, APIs will do in-place encryption/decryption operations.

```
mt_u32 phyaddr;
mt_u8 *viraddr;
mt_u32 buffsize = 0x10000;

phyaddr = (mt_u32)mt_mmz_new(buffsize, 64, NULL, "mmzbuffer");
if (0 == phyaddr) {
    return NULL;
}

viraddr = mt_mmz_map(phyaddr, 0);
if( NULL == viraddr ) {
    mt_mmz_delete(phyaddr);
    return NULL;
}

return viraddr;
```

We have provided API "mt_unf_cipher_malloc" which allocates a continuity physical buffer from MMZ, STBMs can also directly use this API.

- Key format

Input keys are all in MSB-first format. For example, we have a 64 bits key:

1122334455667788 (HEX)

When it is in an input key buffer, its byte order shall be:

{0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88}

5 Notifications for OTP Operations

To avoid accident access (especially write) from software, OTP write operation is locked by hardware mechanism, called OTP firewall. By default, OTP write operation is locked when system boots.

Software shall call OTP APIs provided by SDK that released by Montage-LZ to access OTP. The declaration of OTP write API is listed as below:

MT_S32 MT_UNF_OTP_write(MT_U32 bitaddr, MT_U32 bitlen, MT_U32 data);

OTP firewall disables operation is encapsulated inside the implementation of OTP APIs. For example, in STB production stage, to enable SCS, software calls NOCS API:

csdEnableScs, calls:

MT_UNF_OTP_write.

Inside *MT_UNF_OTP_write*, it disables OTP firewall first, and then sets SCS bits in OTP to enable SCS. After set, software shall call the API to reset the system so that the OTP change can take effect and with the system reset, OTP firewall is enabled again.

6 Hardware Resources Usages

Key Table, Key Ladder, Crypto Engine, RNG, and CERT are all hardware mechanisms. For example, supported algorithms, such as SHA-256, SHA-512, RSA encryption/decryption, are all done by hardware IP, no third-party software libraries are involved.

All hardware features required by NOCS3.2 are encapsulated by security drivers and cipher APIs described above.

7 Explanations of Internal Structures

When implementing NOCS3.2 APIs, a set of internal structures are used. To help STBMs to understand internal implementation of NOCS3.2 APIs, this section explains all the internal structures in details.

Internal structures are listed below:

- SCertResourceHandle
- SSecCipherSession
- SSecStreamSession
- SSecHashContext
- SCsdInitParameters
- SCsdTerminateParameters
- ECsdBootMode
- ECsdScsSize
- SCsdScsPvPathHandle
- SCsdScrKeyPathHandle
- SCsdDscKeyPathHandle
- SCsdR2RKeyPathHandle

7.1 SCertResourceHandle

```
struct SCertResourceHandle {
    int cert_handle;
    unsigned int magic_cert;
};
```

Defined in *ca_cert_impl.h*.

ca_cert_impl.h is the implementation header file of *ca_cert.h*. It defines all internal structures of CERT that listed below.

- *cert_handle* is obtained from cert driver when calling *mt_unf_cert_open()*. Used by driver and hold/released by caller.
- *magic_cert* is a magic data (0xdeadbeaf), and it is set by cert API *certLock()*. Used by driver to check if cert is locked success before operating hardware Cert IP.

7.2 SSecCipherSession

```
struct SSecCipherSession
{
    TUnsignedInt32 magic;
    TBoolean isStreamSession;
    TBoolean isEncryptionSession;
    TUnsignedInt8 keyID[2][16];
    TUnsignedInt8 keySlotID[2];
    TUnsignedInt8 key256SlotID[2];
    TUnsignedInt8 polar;
    size_t metadataSize;
    TUnsignedInt8 metadata[4];
    TUnsignedInt16 EMI;
    TBoolean isKeySet;
    TBoolean isUseFlashProtKey;
    TTransportSessionId transportSessionId;
    TUnsignedInt32 nextKeyId;
    TUnsignedLong cryptoHandle;
    TUnsignedInt8 IV[2][16];
    TUnsignedInt8 keySlotUsed[2];
    TUnsignedInt16 keySlotLatest;
    TUnsignedLong dscHandle;
    TSecPidInfo pidInfo;
    TBoolean isMultiSession;
};
```

Defined in nocs_sec_impl.h.

nocs_sec_impl.h is the implementation header file of nocs_sec.h. It defines all internal structures of SEC that listed below.

- Magic
A constant value 0xdeadbeaf, it used to flag if the session is valid
- isStreamSession
To record whether it is a stream session or RAM2RAM session when creates
- isEncryptionSession
To record whether it is an encryption session or decryption session when creates
- keyID[2][16]
To record key ID transferred from caller
- keySlotID[2]
Two 128 bits key slot IDs requested from drivers
- key256SlotID[2]
Two 256 bits key slot IDs requested from drivers
- Polar
Indicate key polarity, 0 is even, 1 is odd
- metadataSize
The size of NAGRA CERT metadata
- metadata[4]
The metadata of NAGRA CERT
- EMI
To record input encryption method indication
- isKeySet
Indicate if the key is set to keyslot
- isUseFlashProtKey

- To notify drivers if flash protection key is used or not
- `transportSessionId`
To save current `transportSessionId` transferred from caller
- `nextKeyIdx`
indicate index of the next key
- `cryptoHandle`
The handle of crypto engine device which is used in RAM2RAM session
- `IV[2][16]`
To record IV transferred from caller
- `keySlotUsed[2]`
To record which `keySlotID` is currently used: even or odd key
- `keySlotLatest`
Indicate the latest key ID
- `dscHandle`
To record the handle of descrambler
- `pidInfo`
To record the information of PID

```
struct PID_PAIR
{
    TUnsignedInt16 pid;
    TUnsignedInt16 type;
};
typedef struct MT_SEC_PID_INFO_S
{
    TTransportSessionId tsid;
    TUnsignedInt32 sessionType;
    TUnsignedInt32 pidNum;
    struct PID_PAIR pidList[MAX_PID_NUMS];
} TSecPidInfo;
```
- `isMultiSession`
Flag if the session is multiple processes.

7.3 SSecStreamSession

```
struct SSecStreamSession
{
    TUnsignedInt32 magic;
    TBoolean isEncryptionSession;
    TTransportSessionId transportSessionId;
    TUnsignedInt16 EMI;
    TUnsignedInt8 keyID[4][16];
    TUnsignedInt16 keySlotID[4];
    TUnsignedInt8 keySlotStat;
    TUnsignedLong audioDscHandle;
    TUnsignedLong videoDscHandle;
    TUnsignedLong recDscHandle;
    TSecPidInfo pidInfo;
    TUnsignedLong cryptoHandle;
    TBoolean isMultiSession;
};
```

Defined in `nocs_sec_impl.h`.

- **Magic**
A constant value 0xdeadbeaf, it used to flag if the session is valid
- **isEncryptionSession**
To record whether it is an encryption session or a decryption session when creates.
- **transportSessionId**
To save current transportSessionId transferred from caller.
- **keyID[4][16]**
To record keyID transferred from caller
- **keySlotID[4]**
Two key slot IDs requested from drivers
- **keySlotStat**
To flag current should use keySlotID[x]
- **EMI**
To record input encryption method indication
- **audioDscHandle**
To record descramble handle of audio
- **videoDscHandle**
To record descramble handle of vedio
- **pidInfo**
To record the information of PID

```

struct PID_PAIR
{
    TUnsignedInt16 pid;
    TUnsignedInt16 type;
};
typedef struct MT_SEC_PID_INFO_S
{
    TTransportSessionId tsid;
    TUnsignedInt32 sessionType;
    TUnsignedInt32 pidNum;
    struct PID_PAIR pidList[MAX_PID_NUMS];
} TSecPidInfo;

```
- **cryptoHandle**
To record the handle of crypto engine
- **isMultiSession**
Flag if the session is multiple processes.

7.4 SSecHashContext

```

struct SSecHashContext
{
    TUnsignedLong priv_handle;
    TBoolean isUseKeySlot;
    TUnsignedInt32 keySlots[2];
    TUnsignedInt8 metadata[4];
};

```

Defined in nocs_sec_impl.h.

- **priv_handle**
Saved hash handle created from drivers.
- **isUseKeySlot**
The flag of if use hardware key slots. For example: use CERT key.

- keySlots[2]
The ID of key slots only is used for hardware key.
- metadata[4]
The meta data of CERT.

7.5 SCsdInitParameters

```
struct SCsdInitParameters
{
    unsigned int dummy; //dummy member to suppress compiler warnings/errors.
};
```

Defined in nocs_csd_impl.h.

nocs_csd_impl.h is the implementation header file of nocs_csd.h. It defines all internal structures of CSD that listed below.

7.6 SCsdTerminateParameters

```
struct SCsdTerminateParameters
{
    unsigned int dummy; //dummy member, to suppress compiler warnings/errors.
};
```

Defined in nocs_csd_impl.h.

7.7 ECsdBootMode

```
enum ECsdBootMode
{
    CSD_BOOT_SPI_NOR = 0,
    CSD_BOOT_PNAND,
    CSD_BOOT_SPI_NAND,
    CSD_BOOT_SPI_EMMC,
    LAST_CSD_BOOT
};
```

Defined in nocs_csd_impl.h

- CSD_BOOT_SPI_NOR
Boot device is SPI NOR.
- CSD_BOOT_PNAND
Boot device is RAW NAND.
- CSD_BOOT_SPI_NAND
Boot device is SPI NAND.
- CSD_BOOT_SPI_EMMC
Boot device is EMMC
- LAST_CSD_BOOT
Except CSD_BOOT_SPI_NOR, CSD_BOOT_PNAND, CSD_BOOT_SPI_NAND or CSD_BOOT_SPI_EMMC, no other boot devices are supported.

7.8 ECsdScsSize

```
enum ECsdScsSize
{
    CSD_SCS_SIZE_256K = 0x40000,
    CSD_SCS_SIZE_512K = 0x80000,
    CSD_SCS_SIZE_2M = 0x200000,
    CSD_SCS_SIZE_4M = 0x400000,
};
```

Defined in nocs_csd_impl.h, and it is useless now.

7.9 SCsdScsPvPathHandle

```
struct SCsdScsPvPathHandle
{
    unsigned int dummy; //dummy member, to suppress compiler warnings/errors.
};
```

Defined in nocs_csd_impl.h.

7.10 SCsdScrKeyPathHandle

```
struct SCsdScrKeyPathHandle
{
    unsigned int even_slot;
    unsigned int odd_slot;;
};
```

Defined in nocs_csd_impl.h

- even_slot
Even key slot to set even key.
- odd_slot
Odd key slot to set odd key.

7.11 SCsdDscKeyPathHandle

```
struct SCsdDscKeyPathHandle
{
    unsigned int dmx_id;
    unsigned int dsc_handle;
    unsigned int even_slot;
    unsigned int odd_slot;
    unsigned char iv[16];
    unsigned int iv_len;
};
```

Defined in nocs_csd_impl.h

- dmx_id
Descrambler ID, got it when opening descrambler.
- dsc_handle
Descrambler handle created by caller when initializing descrambler.
- even_slot

- Even key slot to set even key.
- odd_slot
Odd key slot to set odd key.
- iv[16]
IV to be set if exists.
- iv_len
IV length.

7.12 SCsdR2RKeyPathHandle

```
struct SCsdR2RKeyPathHandle
{
    unsigned int dummy; //dummy member, to suppress compiler warnings/errors.
};
```

Defined in nocs_csd_impl.h.

7.13 External CSD Montage-LZ API

Declare in nocs_csd_impl.h

- TCsdStatus mtCsdDscKeysOp(TCsdUnsignedInt16 xEmi, TCsdDscKeyPathHandle *pxDscKeyPathHandle);
It should be implemented by STBs, and it is called after csdSetClearTextDscHostKeys and csdSetClearTextDscHostKeys. Usually, you can attach key slots and descrambler in it. But if you do not need it, please implement it with nothing.

7.14 . External SEC Montage-LZ API

Declare in nocs_sec_impl.h

- TSecStatus mtSecOpenStreamDecryptSession(TSecCipherSession xSession);
It should be implemented by STBs, and it is called after secOpenStreamDecryptSession. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecOpenStreamEncryptSession (TSecCipherSession xSession);
It should be implemented by STBs, and it is called after secOpenStreamEncryptSession. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecStreamDecryptSessionOpen (TSecStreamSession xSession);
It should be implemented by STBs, and it is called after secStreamDecryptSessionOpen. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecStreamEncryptSessionOpen (TSecStreamSession xSession);
It should be implemented by STBs, and it is called after secStreamEncryptSessionOpen. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecStreamSessionClose (TSecStreamSession xSession);
It should be implemented by STBs, and it is called after secStreamSessionClose. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecCloseSession(TSecCipherSession xSession);
It should be implemented by STBs, and it is called after secCloseSession. But if you do not need it, please implement it with doing nothing.
- TSecStatus mtSecSetTransportSessionOp (TSecCipherSession xSession);
It should be implemented by STBs, and it is called after setting key API. Usually, you can configure crypto in it. But if you do not need it, please implement it with doing nothing.

- TSecStatus mtSecSetStreamTransportSessionOp (TSecStreamSession xSession);
It should be implemented by STBMs, and it is called after setting stream key API. Usually, you can attach key slots and descrambler in it. But if you do not need it, please implement it with doing nothing.

8 Examples

When starting calling NOCS API to handle data/keys, such as RAM2RAM encryption/decryption, or descrambling streams, some operations must be done first. Here are some examples to guide STBMs to work out all these cases.

8.1 RAM2RAM Encryption/Decryption Example

Code snippet below implements a clear key encrypt session with EMI:

NOCS_EMI_AES128_CBC_ZEROIV_TAIL_CLEAR.

Please note that return value check is ignored for only demonstrating functions.

For decrypt session, call secOpenRam2RamDecryptSession(TSecCipherSession *pxSession).

```

#include "mt_common.h"
#include "mt_unf_cipher.h"
#include "ca_sec.h"
#include "nocs_sec_impl.h"

int main(void)
{
    mt_s32 ret = MT_SUCCESS;

    ret = mt_sys_init();
    ret = mt_unf_cipher_init();

    //allocate 64k bytes buffer
    TUnsignedInt8 *pInBuffer = (TUnsignedInt8 *)secAllocateBuffer(0x10000);
    //in-place encrypt/decrypt
    TUnsignedInt8 *pOutBuffer = pInBuffer;

    //open encrypt session
    TSecCipherSession pSecCipherSession = NULL;
    TSecStatus sret = secOpenRam2RamEncryptSession(&pSecCipherSession);

    TUnsignedInt8 xClearTextKey[16] = { /*Your clear text key*/ };
    sret = secSetClearTextKey(pSecCipherSession, NOCS_EMI_AES128_CBC_ZEROIV_TAIL_CLEAR,
        0, NULL, 16, xClearTextKey);

    TUnsignedInt8 zero_iv[16] = {0x00};
    sret = secSessionEncrypt(pSecCipherSession, pInBuffer, pOutBuffer,
        0x10000, zero_iv, 16);

    //Check the encrypted result

    if (pSecCipherSession != NULL) {
        secSessionClose(pSecCipherSession);
    }
    secFreeBuffer(pInBuffer);

    mt_unf_cipher_deinit();
    mt_sys_deinit();

    return ret;
}

```

8.2 Transport Stream Descrambling Example

To descramble an input transport stream, calling steps are similar, but to call more SDK APIs. Please check snippets below as reference.

Please note that demo code below uses set2LevelProtectedKey. If uses cert key, call secUseCertKey() instead. For more details, please refer related sample code presented by SDK.

```

//TS descramble session
#include "mt_common.h"
#include "mt_unf_dmx.h"
#include "mt_unf_descrambler.h"
/*...other necessary SDK headers*/
#include "mt_unf_cipher.h"
#include "ca_sec.h"
#include "nocs_sec_impl.h

int main(void)
{
    mt_s32 ret = MT_SUCCESS;

    ret = mt_sys_init();
    ret = mt_unf_cipher_init();
    ret = MT_UNF_DMX_Init();
    ret = mtadp_fe_init(); //front end init
    ret = MT_UNF_DMX_AttachTSPort(DMX_ID, MT_UNF_DMX_PORT_TSI_0);
    ret = mtadp_fe_connect(1, frequency, symbol_rate, qam_mode);

    /*More init call to Display, Sound, Video-out...*/

    /*Define your TSID*/

    TSecFunctionTable *pTable = secGetFunctionTable();
    const ISecStreamSession *pStreamSessionF = pTable->secStreamDecryptSession;
    //Create stream session
    TSecStreamSession pSecSession = NULL;
    pStreamSessionF->open(&pSecSession, TSID, NOCS_EMI_TS_DVB_CSA2);
    TUnsignedInt8 keyslot_id[2] = {0};
    pStreamSessionF->getKeySlot(pSecSession, 0, NULL, &keyslot_id[0]);
    pStreamSessionF->getKeySlot(pSecSession, 0, NULL, &keyslot_id[1]);

    //create descrambler
    mt_handle dsc_handle = MT_INVALID_HANDLE;
    MT_UNF_DMX_CreateDescrambler(DMX_ID, &dsc_handle);
    MT_UNF_DMX_SetDescramblerEvenSlot(dsc_handle, keyslot[0]);
    MT_UNF_DMX_SetDescramblerOddSlot(dsc_handle, keyslot[1]);

    /*Process ECM here, and get CipheredEvenContentKey, CipheredOddContentKey,

    L1ProtectKey, L2ProtectKey*/

    //set even key
    TSecKeyParams keyParams = {SEC_KEY_POLARITY_EVEN, FALSE, NOCS_EMI_TS_DVB_CSA2};
    pStreamSessionF->set2LevelProtectedKey(keyslot[0], &keyParams,
        8, CipheredEvenContentKey, 16, L1ProtectKey, L2ProtectKey);

    //set odd key
    keyParams.polarity = SEC_KEY_POLARITY_ODD;
    pStreamSessionF->set2LevelProtectedKey(keyslot[1], &keyParams,
        8, CipheredOddContentKey, 16, L1ProtectKey, L2ProtectKey);

```



```
/*Call other SDK APIs to start descrambling...*/  
  
//all done  
pStreamSessionF->close(pSecSession);  
  
/*Call other SDK APIs to free or close resources used.*/  
  
mt_unf_cipher_deinit();  
mt_sys_deinit();  
  
return ret;  
}
```

9 Acronyms

Name	Description
SOC	System On Chip
SCS	Secure Chipset Startup
STR	Suspend To RAM
RNG	Random Number Generator
CM	Chipset Manufacturer
STBM	Set-Top-Box Manufacturer
SDK	Software Development Kit